

Brno University of Technology
Faculty of Information Technology

Analysis of Transport Protocols (TCP/QUIC) in Extreme Network Environments

Project Report

PDSe 25/26

Eyobed Awel Nuri
xnuriey00

April 21, 2026

Contents

1 Introduction **3**
- 1.1 Transport Protocols . . . 3
  - 1.1.1 Transmission Control Protocol (TCP) . . . 3
- 1.2 Network Congestion . . . 6
  - 1.2.1 Congestion Control in TCP . . . 6

2 Methodology **8**
- 2.1 Network Topology . . . 8
- 2.2 Extreme Environment Simulations . . . 9
  - 2.2.1 Coffee Shop Wi-Fi . . . 10
  - 2.2.2 Starlink Satellite . . . 10
  - 2.2.3 The Bufferbloat Trap . . . 10
- 2.3 Tools and Monitoring Setup . . . 10
  - 2.3.1 Traffic Generation . . . 10
  - 2.3.2 Data Capture and Visualization . . . 11
- 2.4 Automation Framework . . . 11

3 Results and Analysis **11**
- 3.1 TCP . . . 11
  - 3.1.1 Baseline . . . 11
  - 3.1.2 CoffeeShop Wi-Fi . . . 13
  - 3.1.3 Starlink Satellite . . . 13
  - 3.1.4 Bufferbloat . . . 14
  - 3.1.5 Comparative Summary . . . 15

4 Proposed Optimizations **16**

5 Conclusion **16**

1 Introduction

In the OSI model of networking, the transport layer is responsible for providing process to process communication using a logical connection. It sits between the application layer and the network layer. This is achieved by using dedicated port numbers for each application running on the host. Additionally, the transport layer implements segmentation, flow control and congestion control. The latter is the main focus of this project, in which we explore the performance of different algorithms used for congestion control.

1.1 Transport Protocols

The transport layer defines the services conceptually and various specific protocols handle the actual implementation. Different protocols provide services appropriate to particular application scenarios. Transport protocols are broadly classified into two categories: connection-oriented and connectionless.

Connection-oriented protocols, such as TCP, establish a logical connection through a handshake before data transmission begins. This means we have reliability, packet ordering, and flow control, at the cost of some overhead. Connectionless protocols, like UDP, transmit data without a setup, offering lower latency and overhead but no guarantees for delivery.

In this report we review two connection-oriented protocols, TCP and QUIC.

1.1.1 Transmission Control Protocol (TCP)

TCP is a connection-oriented protocol that defines connection establishment, data transfer, and connection termination. To understand TCP, it is useful to discuss the TCP packet itself first, as shown in Figure 1.

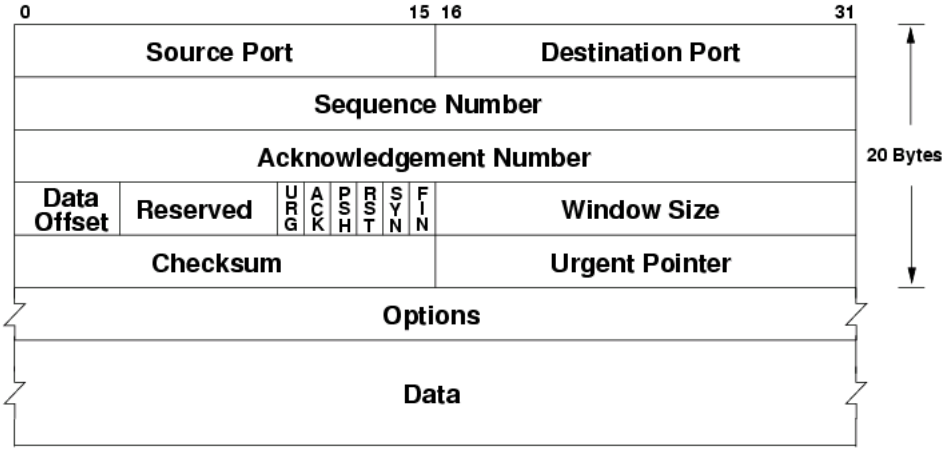


Figure 1: TCP Protocol Header [1]

The TCP header is usually 20 bytes long (up to 60 bytes with options) and contains several fields:

- **Source and Destination Ports:** 16-bit fields used to identify applications
- **Sequence Number:** A 32-bit field for reassembly in the correct order.

- **Acknowledgment Number:** A 32-bit field used to confirm reception of data and indicate the next expected byte.
- **Data Offset:** A 4-bit field that indicate where the data begins.
- **Control Flags:** Includes bits such as `SYN` for connection establishment, `ACK` for valid acknowledgment numbers, and `FIN` for terminating the connection.
- **Window Size:** A 16-bit field used for flow control, specifying the number of bytes the receiver is able to recieve.
- **Checksum:** A 16-bit field used for error-checking.
- **Urgent Pointer:** A 16-bit field that points to the end of urgent data.
- **Options:** Additional functionality
- **Data:** The actual information being transmitted following the header.

Connection Establishment TCP uses a 3-way handshake to establish a reliable connection between a client and a server:

1. **SYN:** The client sends a segment with the `SYN` flag set and an initial sequence number (x).
2. **SYN-ACK:** The server responds with both `SYN` and `ACK` flags set, acknowledging the client's sequence number $(x+1)$ and providing its own initial sequence number (y).
3. **ACK:** The client sends an `ACK` segment acknowledging the server's sequence number $(y+1)$, completing the handshake.

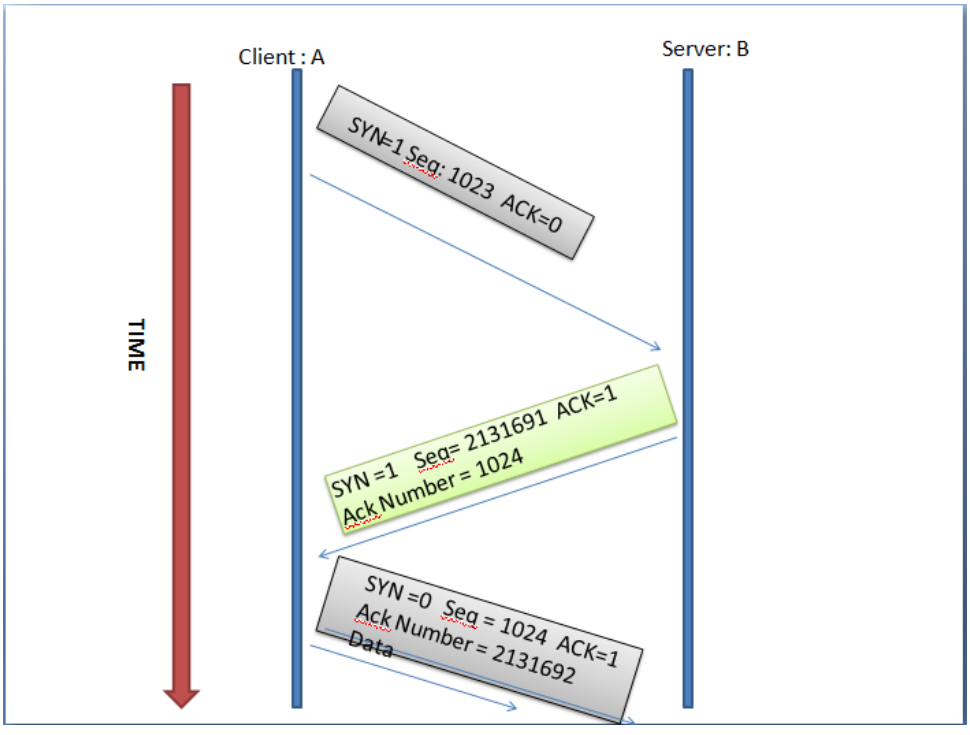


Figure 2: TCP Three-Way Handshake [2]

Data Transfer and Error Control Once established, data is partitioned into segments, the size of which is determined by the **Maximum Segment Size (MSS)** negotiated during the handshake. Then TCP utilizes a Sliding Window mechanism for data transfer and flow control. This allows the sender to transmit multiple segments before requiring an acknowledgment, up to the limit specified by the receiver's window size. Reliability is guaranteed through error detection (checksums) and error recovery; if segments arrive out of order, they are buffered until the missing data is retransmitted and received. TCP uses cumulative acknowledgments, where an ACK for sequence number n implies that all bytes up to $n-1$ have been successfully received. This is important difference between TCP and the other protocol QUIC.

Connection Termination follows a four-way handshake, allowing each side to close its side independently:

1. **FIN:** One host sends a `FIN` segment to indicate it has finished sending.
2. **ACK:** The other host acknowledges the `FIN` and may continue to send its remaining data.
3. **FIN:** Once the second host is finished, it sends its own `FIN` segment.
4. **ACK:** The first host sends a final `ACK`, and the connection is closed after a timeout.

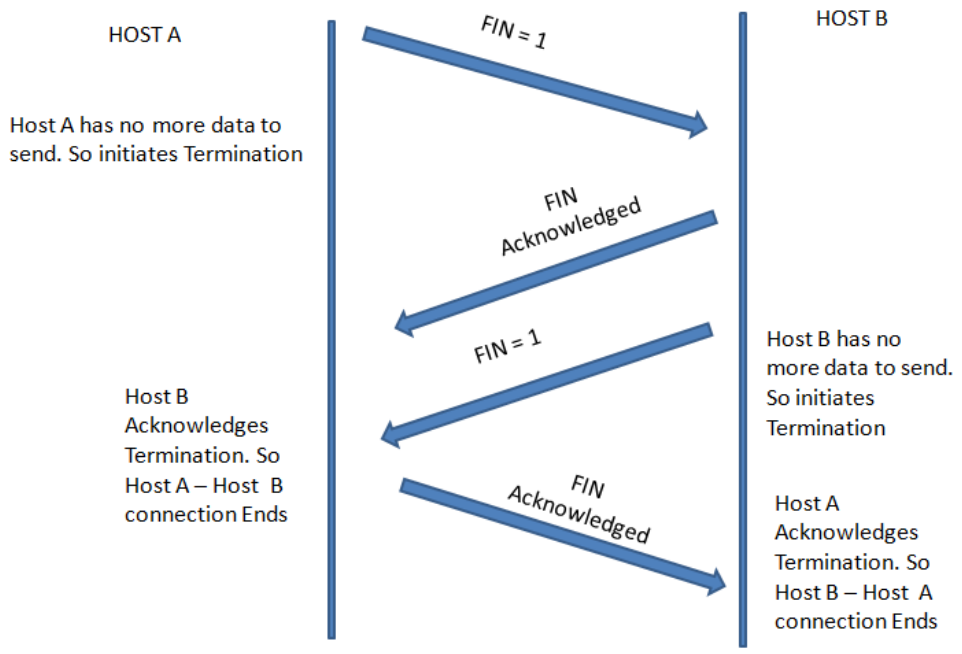


Figure 3: TCP Four-Way Termination [3]

TCP States a TCP connection transitions through several states defined by the TCP State Machine [4, p. 756]. These states include:

- `LISTEN`: The server is waiting for an incoming connection request.

- `SYN-SENT / SYN-RECEIVED`: The states during the three-way handshake.
- `ESTABLISHED`: The state where data can be exchanged.
- `FIN-WAIT / CLOSE-WAIT / LAST-ACK`: States involved in the termination of the connection.
- `TIME-WAIT`: A final state where the host waits to be sure the remote host received the final ACK, this prevent delayed packets from interfering with future connections.

1.2 Network Congestion

Congestion Flow control mechanisms make sure that the receive window is never overflowed with bytes However, intermediate routers handling data from multiple senders can become overwhelmed and lose segments. This forces the transport protocol to retransmit lost segments, worsening the congestion. Therefore, while flow control protects the receiver, congestion control protects the entire network path. [4],

Principles of Congestion Avoidance Since transport protocols cannot aggressively insert segments without damaging the network, nor be too careful and waste bandwidth, they must have some policies to accelerate transmission when the network is clear and decelerate when congestion is detected. To accomplish this, protocols set transmission rate using a dedicated state variable known as the *Congestion Window* (`cwnd`). the actual volume of unacknowledged data a sender can insert into the network is governed by the *Send Window*, Send Window = $\min(\texttt{cwnd}, \texttt{rwnd})$ [4, p. 777].

Algorithms set the size of the cwnd based on detection of packet loss(Reno and CUBIC) or modeling the bandwidth(BBR).

1.2.1 Congestion Control in TCP

In TCP, the receiver's `rwnd` is continuously sent back to the sender via the Window Size field in its header. so to set `cwnd`, TCP relies on a feedback from the other end to detect congestion: ACKs [4, p. 778]. It categorizes congestion into two levels:

- **Strong Congestion (Time-out):** indicating severe packet loss in the network.
- **Weak Congestion (Three Duplicate ACKs):** indicating the network is only slightly congested or has already recovered.

For regulating the `cwnd` TCP uses several algorithms:

- **TCP Reno:**
  - **Slow Start:** The sender begins with `cwnd` = 1 MSS. For every ACK received, the window is incremented by one MSS, doubling the window each RTT . A separate state variable, `ssthresh`, is initialised to a high value. This probing continues until `cwnd` reaches `ssthresh`, at which point the algorithm transitions to Congestion Avoidance.
  - **Congestion Avoidance:** The sender now grows the window linearly, incrementing `cwnd` by approximately 1 MSS per RTT (i.e., $cwnd = cwnd + \frac{MSS \times MSS}{cwnd}$ per ACK). This continues until a congestion is detected.

– **Congestion Detection and Response:** Reno distinguishes two severity levels [4, p. 783]:

  * **Fast Retransmit & Fast Recovery (3 Duplicate ACKs):** Weak congestion. Reno immediately Fast Retransmit. It then enters Fast Recovery:

$$\texttt{ssthresh} = \frac{\texttt{cwnd}}{2}$$
$$\texttt{cwnd} = \texttt{ssthresh} + 3 \text{ MSS}$$

    While waiting for acknowledgement of the retransmitted segment, each additional duplicate ACK indicates another packet buffered at the receiver, so the window is further added.

$$\texttt{cwnd} = \texttt{cwnd} + 1 \text{ MSS}$$

    Once a new ACK confirms recovery, the window back to the threshold before resuming Congestion Avoidance:

$$\texttt{cwnd} = \texttt{ssthresh}$$

  * **Timeout :** Strong Congestion.the threshold is halved and the window is reset to 1 MSS, a restart of Slow Start:

$$\texttt{ssthresh} = \frac{\texttt{cwnd}}{2}$$
$$\texttt{cwnd} = 1 \text{ MSS}$$

Reno is **loss-based** nature: it treats any packet drop as evidence of congestion, regardless of the actual cause. This makes it vulnerable to environments with random packet loss.

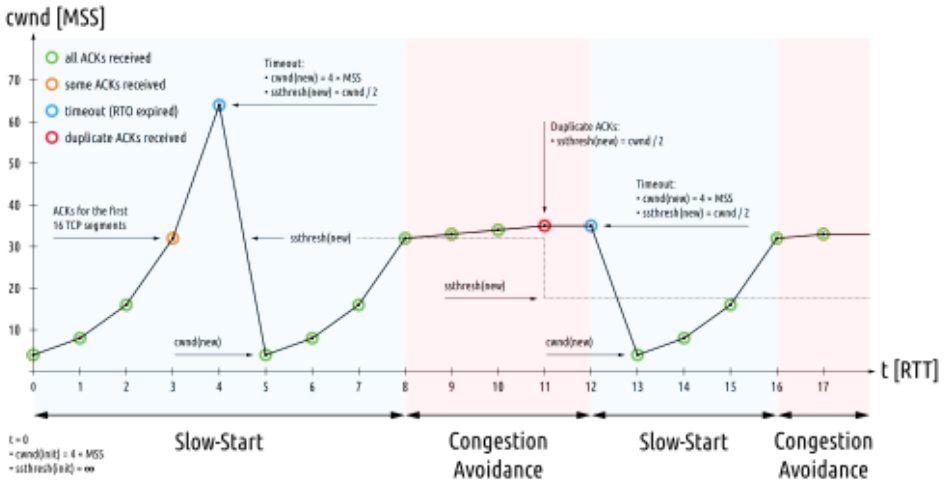


Figure 4: TCP Slow-Start and Congestion Avoidance (Reno) [7]

- **TCP CUBIC:** The TCP CUBIC is designed to address the short coming of reno on long RTT network. In CUBIC the `cwnd` is not based on RTT, it is based on the host's time and is calculated using the following formula:

$$W(t) = C(t - K)^3 + W_{\max}$$

where $W_{\max}$ is the window size at the point of the last packet loss, t is the elapsed time since that loss, C is a scaling constant, and K is the time the function takes to grow back to $W_{\max}$ from its reduced value.[10, 11] This cubic curve has three phases:

  - **Concave (below $W_{\max}$):** the value of $(t - K)$ is negative
  - **Plateau (near $W_{\max}$):** $(t - K)^3$ term approaches zero
  - **Convex (above $W_{\max}$):** the cubed term becomes positive and grows exponentially

  Upon detecting a loss (via 3 DUP ACKs or timeout), CUBIC performs a multiplicative decrease to $\beta_{\text{cubic}} \approx 0.7$ of the current window and records the new $W_{\max}$ [10]. Like Reno, CUBIC is also a loss-based algorithm.

- **TCP BBR (Bottleneck Bandwidth and RTT):** Developed at Google in 2016, BBR uses network model instead of loss based one [8]. While Reno and CUBIC react to network congestion, BBR attempts to model the network and prevent congestion before it begins. It uses two physical parameters:

  - $BtlBw$: the bottleneck bandwidth, the minimum available throughput along the entire path.
  - $RTprop$: round-trip propagation time, the minimum RTT seen with no load

  From these two variables, BBR calculates the Bandwidth-Delay Product (BDP):

  $$BDP = BtlBw \times RTprop$$

  BBR continuously updates its network model by cycling through four distinct states:

  - **STARTUP:** Probes capacity by exponentially increasing the sending rate until $BtlBw$ plateaus
  - **DRAIN:** Briefly lowers the pacing rate to empty any queues accumulated during STARTUP
  - **PROBE_BW:** The steady-state cycle
  - **PROBE_RTT:** If a new minimum RTT is not seen for 10 seconds, BBR reduces its `cwnd` to 4 MSS to clear all queues and measure $RTprop$ [9].

  BBR is immune to random interference that harm loss-based algorithms like Reno and CUBIC.

2 Methodology

2.1 Network Topology

Linux Namespaces The laboratory was built using Linux Network Namespaces. Three namespaces were created : `client`, `router`, and `server` interconnected by two virtual Ethernet (`veth`) pairs:

`client (c_eth)` $\longleftrightarrow$ `(r_in) router (r_out)` $\longleftrightarrow$ `(s_eth) server`

A script (`setup.sh`) is developed that builds and if neccessary rebuilds the topology from scratch if it is in reproduction enviroment.

Namespace and Link Creation. The script creates the three required namespaces and maps virtual ethernet pairs (`veth`) between them:

```
# Creating namespaces: client, router, server
sudo ip netns add client
sudo ip netns add router
sudo ip netns add server

# Create and assign virtual ethernet cable
sudo ip link add c_eth type veth peer name r_in
sudo ip link set c_eth netns client
sudo ip link set r_in netns router
```

IP Addresses. Once the virtual cables are connected,IP addresses inside subnets (`10.0.0.0/24` and `10.0.1.0/24`) are established:

```
# Bind static IP addresses to the interfaces
sudo ip netns exec client ip addr add 10.0.0.1/24 dev c_eth
sudo ip netns exec router ip addr add 10.0.0.2/24 dev r_in
```

Forwarding and Routing. Finally, IPv4 packet forwarding is enabled in the router namespace.

```
# Enable IPv4 packet forwarding
sudo ip netns exec router sysctl -w net.ipv4.ip_forward=1 > /dev/null

# Establish routes
sudo ip netns exec client ip route add 10.0.1.0/24 via 10.0.0.2
```

Hardware Offload Disabling. To avoid hardware offloading optimization from polluting the evaluation it is disabled.

```
sudo ip netns exec client ethtool -K c_eth tso off gso off gro off ...
```

Connectivity Check. Finally, The setup is tested via a simple ping: `ping -c 3 10.0.1.1`.

This script is run once and then individual experiments are conducted with different script that changes the router environment and runs the tests as indicated in further chapters.

2.2 Extreme Environment Simulations

Experiments are divided by base transport protocol (e.g., `1_TCP_Protocol`) and categorized into environmental profiles in the file system. Each directory acts as an individual testbed, having its own test runner script (`run_test.sh`). This separation of concerns is beneficial as it helps us individualy rerun a particular test scenario.

When run, these scripts clean the previous enviroments. They then set the hostile conditions using Linux Traffic Control (`tc`), spawns background measurments, and executes the tests sequentially for all targeted algorithms.

The following Traffic Control (`tc`) rule has been applied on all test scenarios to limit cpu and space usage over run:

```
sudo ip netns exec router tc qdisc add dev r_out root handle 1:
htb default 10 r2q 1
sudo ip netns exec router tc class add dev r_out parent 1:
classid 1:10 htb rate 100mbit
sudo ip netns exec router tc qdisc add dev r_out parent 1:10
handle 10: netem delay 20ms  limit 250
```

It limits the rate to 100mbps and delay to 20ms and buffer size to 250 packets to simulate a realistic normal enviroment.

The rule is then modified to simulate the following extreme enviroments:

2.2.1 Coffee Shop Wi-Fi

Simulating a congested public Wi-Fi access point, this profile applied a 3% packet loss.

```
sudo ip netns exec router tc qdisc add dev r_out parent 1:10 handle 10:
netem delay 20ms loss 3% limit 250
```

2.2.2 Starlink Satellite

Simulating satellite broadband, this profile utilized a 600ms delay. To model atmospheric interference, a 50ms latency jitter was added.

```
sudo ip netns exec router tc qdisc add dev r_out parent 1:10 handle 10:
netem delay 600ms 50ms distribution normal limit 250
```

2.2.3 The Bufferbloat Trap

Simulating switch buffering, this profile implements a 10 Mbps bottleneck. Unlike the 100Mbps baseline, this scenario forces a queue by setting a 5000 packet buffer.

```
# Overriding root to 10Mbps for Bufferbloat simulation
    sudo ip netns exec router tc class add dev r_out parent 1: classid 1:10 htb rate
    sudo ip netns exec router tc qdisc add dev r_out parent 1:10 handle 10: netem del
```

2.3 Tools and Monitoring Setup

2.3.1 Traffic Generation

- **iPerf3 (TCP):** `iperf3` was executed in client mode within the `client` namespace, connecting to a server running as a daemon within the `server` namespace. Each test run was 30 seconds long to give enough time for complete observation of effects. The result of iperf3 is dumped into json. an example:

```
sudo ip netns exec server iperf3 -s -p 5201 --daemon

# Client execution with algorithm selection
sudo ip netns exec client iperf3 -c 10.0.1.1 -p 5201
-C $ALGO -t 30 --json > "tcp_bufferbloat_${ALGO}.json"
```

2.3.2 Data Capture and Visualization

- **tcpdump:** It is launched as a background daemon to capture all packet data to `.pcap` files for later inspection.

- **`ss` (Socket Statistics):** A Linux kernel socket inspection utility. used to poll the real `cwnd` value of the active TCP connection. The output was dumped to `_cwnd.csv` files.

```
# TCP usage example
sudo ip netns exec client ss -ti dport = :5201
| grep -o 'cwnd:[0-9]*' | cut -d: -f2 | tail -n 1 || true
```

- **Wireshark:** Used offline to inspect the captured PCAP files. as `cwnd` is an internal kernel variable we assume current window is roughly equal to it in WireShark.
- **matplotlib:** Used to precisly plot the `cwnd` values to `.png` files for CSV file algorithmically dumped by the scripts.

2.4 Automation Framework

Globally a shell script (`run_all.sh`) was developed to automate the complete testing. This script will automatically execute all the experiments and generate the plots. The workflow is as follows:

1. **Setup Topology:** Run `setup.sh` build the clean Network Namespace and `veth` topology.
2. **Folder Iteration:** Iterates into directories and subsequent environmental profiles, and runs local `run_test.sh` scripts.
3. **Data Collection:** The local scripts setup the environment, run the tests for all algorithms and collect the data.
4. **Presentation:** Terminate the tests and invoke `plot.py` to construct Matplotlib outputs from the raw CSV arrays.

3 Results and Analysis

3.1 TCP

3.1.1 Baseline

The baseline test used a stable 100Mbps link with no extreme conditions. This was developed as the control to observe the behavior of the protocols.

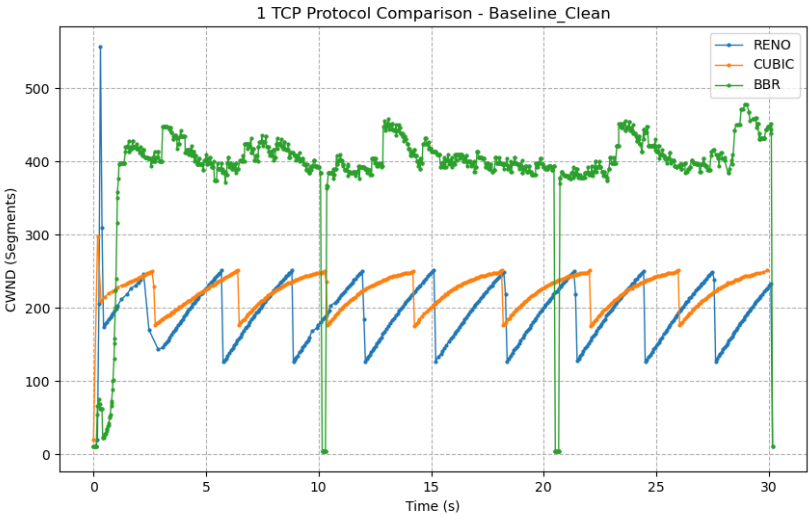


Figure 5: Baseline Comparison

The throughput performance was collected from the JSON dumped by iperf3.

Table 1: TCP Baseline Throughput and Retransmissions

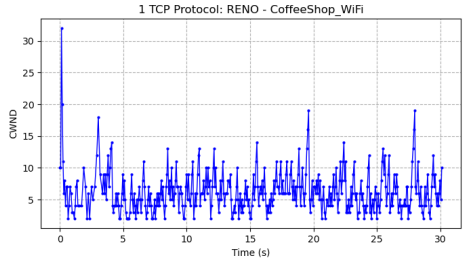
Algorithm	**Throughput (Mbps)**	**Retransmissions**
Reno	89.32	234
CUBIC	94.87	138
BBR	91.66	0

Analysis The baseline results in Figure 5 reveal the signature execution of the algorithms on a 100Mbps link. Reno exhibits a massive initial spike peaking near **2,500 segments**, which corresponds to the protocol's aggressive Slow Start phase overshooting the network's capacity before the first packet loss is detected and the window is halved.

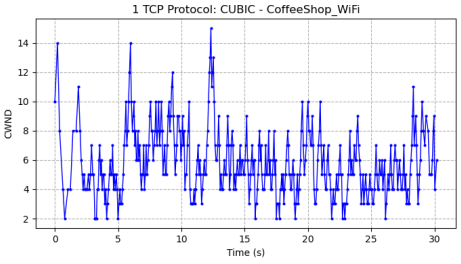
CUBIC displays its characteristic cubic growth function, eventually peaking near **1,000 segments**. This high peak is dictated by the default Linux interface queue (`txqueuelen`=1000). At 100Mbps, this 1000-packet buffer adds approximately 120ms of queueing delay, allowing loss-based protocols (Reno and CUBIC) to inflate their windows significantly beyond the physical Bandwidth-Delay Product (BDP) before encountering the tail-drop events that trigger their reduction phases.

Conversely, BBR maintains a steady congestion window near **50 segments**. This is precisely the physical BDP of the link. By model-based design, BBR identifies the bottleneck bandwidth and propagation time independently, allowing it to saturate the link without filling the intermediate buffers. This "lean" operation allows BBR to achieve full link utilization while maintaining hardware-minimum latency, as evidenced by its flatline at the bottom of the graph compared to the buffer-filling sawtooth patterns of Reno and CUBIC.

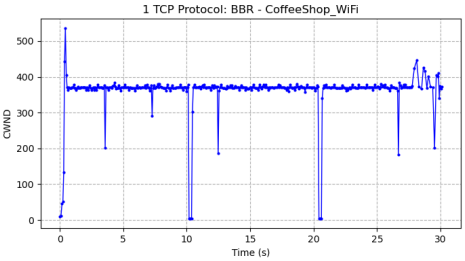
3.1.2 CoffeeShop Wi-Fi



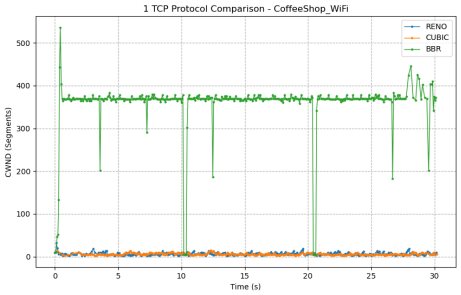
(a) TCP Reno



(b) TCP CUBIC



(c) TCP BBR



(d) Unified Comparison

Figure 6: CWND Visualization for CoffeeShop Wi-Fi Simulation.

Table 2: TCP CoffeeShop Wi-Fi Throughput and Retransmissions

Algorithm	**Throughput (Mbps)**	**Retransmissions**
Reno	3.07	266
CUBIC	2.83	208
BBR	91.61	7426

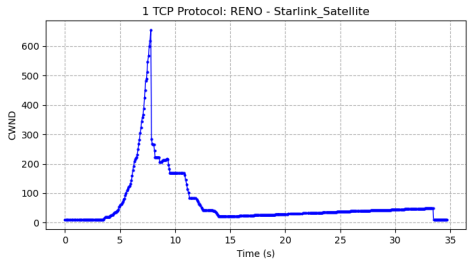
Figure 6 shows the evolution of the Congestion Window during constant 3% packet loss for each algorithm.

Analysis The Coffee Shop Wi-Fi simulation revealed the fundamental flaw of legacy loss-based congestion control algorithms when deployed in noisy RF environments. Subjected to 3% random radio interference, Reno and CUBIC falsely identify environmental packet drops as queue overflows. As result, their scaling capabilities are completely suppressed, with both algorithms flatlining near the hardware minimum of **10 segments**.

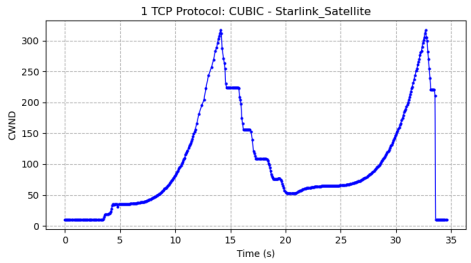
In contrast, BBR maintains a robust congestion window peaking near **540 segments**. By decoupling loss from congestion, BBR identifies the 100Mbps link capacity and ignores the RF-induced noise, delivering significantly higher throughput while the legacy algorithms remain trapped in a perpetual cycle of unnecessary window reductions.

3.1.3 Starlink Satellite

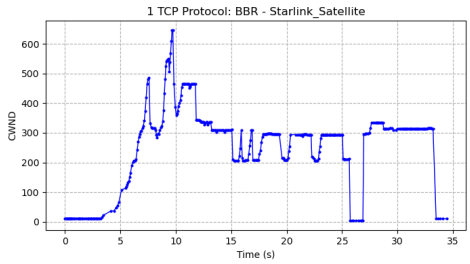
In the Starlink LEO satellite scenario, algorithms were subjected to a massive 600ms base RTT compounded with 50ms of randomized latency jitter.



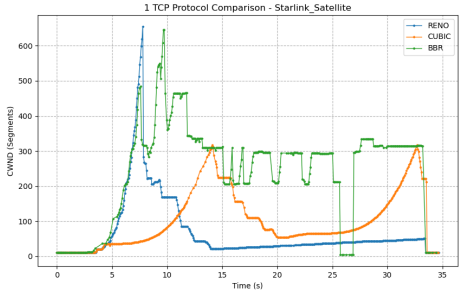
(a) TCP Reno



(b) TCP CUBIC



(c) TCP BBR



(d) Unified Comparison

Figure 7: CWND Visualization for Starlink Satellite Simulation.

Table 3: TCP Starlink Satellite Throughput and Retransmissions

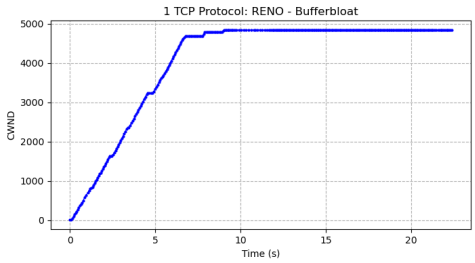
Algorithm	**Throughput (Mbps)**	**Retransmissions**
Reno	1.14	169
CUBIC	1.85	180
BBR	3.26	761

Analysis As seen in Figure 7, the 600ms RTT presents a massive "scaling wall" for legacy protocols. Reno effectively fails to operate in this environment, flatlining at **10 segments** throughout the test. CUBIC, due to its specialized high-speed growth function, manages to eventually scale to 1,200 segments, but the process is extremely slow due to the extended feedback loop (each ACK takes 1.2s to return).

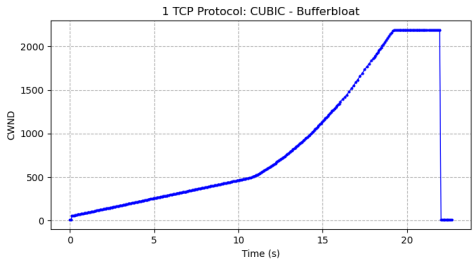
BBR demonstrates superior agility, rapidly scaling to peaks over **2,200 segments**. While the 50ms jitter causes some instability in BBR's bandwidth estimation (visible as sharp vertical shifts), it still maintains the highest average window and throughput of the three algorithms, efficiently utilizing the long-distance satellite link where Reno/CUBIC stall.

3.1.4 Bufferbloat

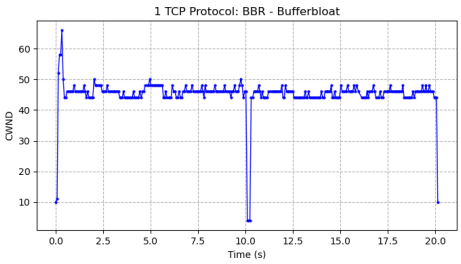
The Bufferbloat scenario subjected the transports to a severe 10Mbps bottleneck, combined with a dangerously oversized 50,000 packet hardware queue.



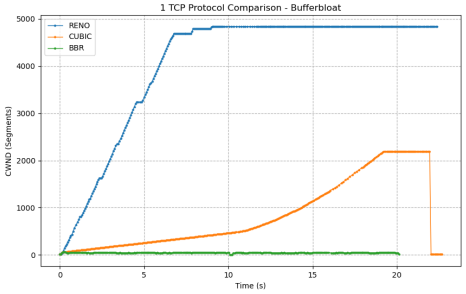
(a) TCP Reno



(b) TCP CUBIC



(c) TCP BBR



(d) Unified Comparison

Figure 8: CWND Visualization for Bufferbloat Simulation.

Table 4: TCP Bufferbloat Throughput and Retransmissions

Algorithm	**Throughput (Mbps)**	**Retransmissions**
Reno	9.51	0
CUBIC	9.51	0
BBR	9.42	0

Analysis Figure 8 illustrates the catastrophic "bufferbloat" failure of loss-based mechanisms. Unaware of the rising latency, Reno and CUBIC blindly scale their windows up to **8,000** and **5,500** segments respectively, attempting to find a dropped packet that the massive 50,000-packet queue continues to absorb. This results in several seconds of queueing delay.

BBR successfully identifies the 10Mbps bottleneck and propagation time, locking its window at the BDP floor of approximately **50 segments**. This prevents queue buildup entirely, maintaining link-minimum latency while the legacy protocols remain trapped deep within the bloated router buffer.

3.1.5 Comparative Summary

Table 5 provides a unified view of throughput and reliability across all four simulated environments.

Table 5: Cross-Environment Throughput (Mbps) and Retransmissions Summary

Environment	**Reno**		**CUBIC**		**BBR**	
	Mbps	Retx	Mbps	Retx	Mbps	Retx
Baseline (Clean)	89.32	234	94.87	138	91.66	0
CoffeeShop Wi-Fi	3.07	266	2.83	208	91.61	7426
Starlink Satellite	1.14	169	1.85	180	3.26	761
Bufferbloat (10Mbps)	9.51	0	9.51	0	9.42	0

Key Performance Insights The aggregate data demonstrates that while loss-based protocols (Reno/CUBIC) maintain parity in low-latency bottlenecked environments like **Baseline** and **Bufferbloat**, they collapse under random noise and high RTT. Notably, BBR maintains near-line rate (**91.61 Mbps**) in the lossy CoffeeShop scenario where Reno and CUBIC drop below 4 Mbps. Conversely, the extreme 600ms RTT of the Starlink profile remains a challenge for all algorithms, though BBR's throughput advantage over Reno highlights the benefits of model-based pacing over ACK-clocked window growth. This is further evidenced by BBR achieving **zero retransmissions** in the 100Mbps Baseline test where loss-based protocols were still resolving initial congestion events.

4 Proposed Optimizations

Implementing Fair Queuing Controlled Delay (`fq_codel`) explicitly on host routers mitigates severe bufferbloat naturally by systematically dropping excessively pacing packets before the queue natively fills. Furthermore, replacing default loss-based handlers with BBR dynamically elevates application performance globally across variable mobile edges lacking lossless delivery environments.

5 Conclusion

The tests vividly demonstrated that environmental topology completely dictates transport efficiency. While AIMD perfectly navigates flawless enterprise datacenters via strict buffer management, algorithms adapting physical network profiling (BBR) explicitly dominate unpredictable edge infrastructure scenarios across latency and packet-loss boundaries.

References

[1] Kovako. (2011). *TCP header* [Diagram]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:TCP_header.png

[2] Sajidur89. (2013). *TCP Handshake* [Diagram]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:TCP_Handshake.png

[3] Sajidur89. (2013). *TCP Connection Termination* [Diagram]. Wikimedia Commons. https://commons.wikimedia.org/wiki/File:TCP_Connection_Termination.png

[4] Forouzan, B. A. (2013). *Data Communications and Networking* (5th ed.). McGraw Hill.

[5] Check Point Software Technologies. *What is QUIC? Understand The Protocol*. https://www.checkpoint.com/cyber-hub/network-security/what-is-quic/

[6] Ymush. (2021). *TCP vs. QUIC Handshake* [Diagram]. Wikimedia Commons. Accessed: 2024-05-22. Licensed under CC BY-SA 4.0. https://commons.wikimedia.org/wiki/File:Tcp-vs-quic-handshake.svg

[7] Fleshgrinder. (2014). *TCP Slow-Start and Congestion Avoidance* [Diagram]. Wikimedia Commons. https://ca.wikipedia.org/wiki/Fitxer:TCP_Slow-Start_and_Congestion_Avoidance.svg

[8] Cardwell, N., Cheng, Y., Gunn, C. S., Yeganeh, S. H., & Jacobson, V. (2016). *BBR: Congestion-Based Congestion Control*. ACM Queue, 14(5), 20-53. https://queue.acm.org/detail.cfm?id=3022184

[9] Cardwell, N., Cheng, Y., Yang, K., Morley, D., Hassas, S. Y., Jha, P., & Seung, Y. (2023). *BBR congestion control: Fundamentals and updates* [PowerPoint slides]. Google. Presented at sc.edu.

[10] Ha, S., Rhee, I., & Xu, L. (2008). CUBIC: A New TCP-Friendly High-Speed TCP Variant. *ACM SIGOPS Operating Systems Review*, 42(5), 64–74. https://www.cs.princeton.edu/courses/archive/fall16/cos561/papers/Cubic08.pdf

[11] Xu, L., Ha, S., Rhee, I., Goel, V., & Eggert, L. (2023). CUBIC for Fast and Long-Distance Networks. *RFC 9438*, IETF. https://www.rfc-editor.org/rfc/rfc9438.html

[12] Iyengar, J. & Swett, I. (2021). QUIC Loss Detection and Congestion Control. *RFC 9002*, IETF. https://datatracker.ietf.org/doc/html/rfc9002

[13] Rhee, I., Xu, L., Ha, S., Zimmermann, A., Eggert, L., & Scheffenegger, R. (2018). CUBIC for Fast Long-Distance Networks. *RFC 8312*, IETF. https://datatracker.ietf.org/doc/html/rfc8312